

RWA Tokenization Platform

Final Project Report

Blockchain Technologies 2 — Option C

Team members:

Sayakhat Shauzat

Ali Kurmantayev

Zere Boranbay

1. Project Overview

Our team built a Real World Asset (RWA) tokenization protocol on Ethereum. The system has these parts:

1. Asset-backed ERC-20 token with role-gated minting
2. Upgradeable contract using UUPS proxy pattern (V1 to V2).
3. ERC-4626 yield vault for token holders.
4. Constant-product AMM ($x*y=k$) with 0.3% fee, written from scratch.
5. Chainlink price oracle with staleness check.
6. Factory using both CREATE and CREATE2.
7. ERC-721 Issuer License NFT.
8. DAO governance with OpenZeppelin Governor and 2-day Timelock.
9. Inline Yul assembly library with Solidity benchmark.

2. GitHub Repository

The full project is on GitHub. The repository contains all smart contracts, tests, frontend, subgraph, deploy scripts, CI workflow and documentation.

 AlleSH419 and  zerk228 fix(ci): install foundry libs and simplify workflow		18e4eaf · 1 minute ago	 29 Commits
 .github/workflows	fix(ci): install foundry libs and simplify workflow	1 minute ago	
 .vscode	chore: init foundry project layout	last month	
 contracts	feat(rwa): RWA V2 with pausable transfers	2 weeks ago	
 docs	docs: coverage report	4 hours ago	
 frontend	feat(frontend): MetaMask connect, balances, swap, vote, sub...	3 days ago	
 script	script: deploy and post-deploy verification scripts	last week	
 subgraph	feat(subgraph): manifest, schema and AssemblyScript mapping	last week	
 test	test(fork): mainnet fork against USDC, Chainlink, UniV2	last week	
 .env.example	chore: init foundry project layout	last month	
 .gitignore	chore: init foundry project layout	last month	
 .solhint.json	chore: init foundry project layout	last month	
 INSTALL.md	docs: README, architecture, deployment, proposal, slides, ins...	2 days ago	
 README.md	docs: README, architecture, deployment, proposal, slides, ins...	2 days ago	
 fmt.toml	chore: init foundry project layout	last month	
 foundry.toml	chore: init foundry project layout	last month	
 remappings.txt	chore: init foundry project layout	last month	

3. Team Commits

All three team members have commits in the project history. The commits use Conventional Commit messages (feat, fix, test, docs, chore, ci).



4. Smart Contract Tests

We use the Foundry framework. The project has 104 tests in total:

80+ unit tests covering all public functions.

10+ fuzz tests for AMM, vault, governance.

3 invariant tests for AMM.

3 fork tests against real mainnet contracts.

All tests pass.

```
PROBLEMS 56 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
LEGION@DESKTOP-A69TSMR MINGW64 ~/Desktop/bch2final (main)
$ forge test -vv
[·] Compiling...
No files changed, compilation skipped

Ran 3 tests for test/AccessControlCase.t.sol:AccessControlCaseTest
[PASS] test_AdminCanRevokeRoleFromAttacker() (gas: 92272)
[PASS] test_AttackerCannotMint() (gas: 20298)
[PASS] test_AttackerCannotUpgrade() (gas: 1376947)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.16ms (2.22ms CPU time)

Ran 3 tests for test/Fork.t.sol:ForkTest
[PASS] test_Fork_ChainlinkETHFeed() (gas: 3783)
[PASS] test_Fork_USDC_Symbol() (gas: 3850)
[PASS] test_Fork_UniV2RouterFactory() (gas: 3785)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 3.57ms (1.20ms CPU time)

Ran 3 tests for test/Reentrancy.t.sol:ReentrancyCaseStudyTest
[PASS] test_NonReentrantDepositProtects() (gas: 48184)
[PASS] test_NonReentrantWithdrawProtects() (gas: 50581)
[PASS] test_ReentrancyGuard_BlocksReenter() (gas: 325863)
Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 4.01ms (1.48ms CPU time)

Ran 6 tests for test/IssuerNFT.t.sol:IssuerNFTTest
[PASS] test_IssueIncrementsId() (gas: 114545)
[PASS] test_MultipleIssues() (gas: 189451)
[PASS] test_NameSymbol() (gas: 17895)
[PASS] test_RevertWhen_NonMinterIssues() (gas: 13775)
[PASS] test_RevokeBurnsToken() (gas: 94624)
[PASS] test_SupportsInterface() (gas: 8170)
Suite result: ok. 6 passed; 0 failed; 0 skipped; finished in 4.35ms (3.41ms CPU time)

Ran 12 tests for test/GovToken.t.sol:GovTokenTest
[PASS] test_AdminCanMint() (gas: 59249)
[PASS] test_AdminGrantsMinter() (gas: 90938)
[PASS] test_DelegateSelf() (gas: 120586)
[PASS] test_DelegateToOther() (gas: 125973)
[PASS] test_DomainSeparatorNonZero() (gas: 5677)
[PASS] test_InitialSupplyToAdmin() (gas: 12909)
[PASS] test_NameSymbol() (gas: 17897)
[PASS] test_NoncesStartAtZero() (gas: 9991)
```

5. Invariant Tests

Three invariant properties run 64 fuzz campaigns with 2048 calls each:

- k value never decreases on AMM swaps.
- LP token total supply stays positive.
- AMM reserves always match real token balances.

All three invariants hold with zero reverts and zero discards.

```
PROBLEMS 56 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
LEGION@DESKTOP-A69TSMR MINGW64 ~/Desktop/bch2final (main)
$ forge test -vv
[PASS] invariant_KNeverDecreasesPerLP() (runs: 64, calls: 2048, reverts: 0)

+-----+-----+-----+-----+-----+
| Contract | Selector | Calls | Reverts | Discards |
+-----+-----+-----+-----+-----+
| AMMHandler | addLiq | 660 | 0 | 0 |
+-----+-----+-----+-----+-----+
| AMMHandler | swap0 | 677 | 0 | 0 |
+-----+-----+-----+-----+-----+
| AMMHandler | swap1 | 711 | 0 | 0 |
+-----+-----+-----+-----+-----+

[PASS] invariant_LPSupplyPositive() (runs: 64, calls: 2048, reverts: 0)

+-----+-----+-----+-----+-----+
| Contract | Selector | Calls | Reverts | Discards |
+-----+-----+-----+-----+-----+
| AMMHandler | addLiq | 708 | 0 | 0 |
+-----+-----+-----+-----+-----+
| AMMHandler | swap0 | 664 | 0 | 0 |
+-----+-----+-----+-----+-----+
| AMMHandler | swap1 | 676 | 0 | 0 |
+-----+-----+-----+-----+-----+

[PASS] invariant_ReservesMatchBalances() (runs: 64, calls: 2048, reverts: 0)

+-----+-----+-----+-----+-----+
| Contract | Selector | Calls | Reverts | Discards |
+-----+-----+-----+-----+-----+
| AMMHandler | addLiq | 694 | 0 | 0 |
+-----+-----+-----+-----+-----+
| AMMHandler | swap0 | 667 | 0 | 0 |
+-----+-----+-----+-----+-----+
| AMMHandler | swap1 | 687 | 0 | 0 |
+-----+-----+-----+-----+-----+

Suite result: ok. 3 passed; 0 failed; 0 skipped; finished in 518.91ms (1.52s CPU time)

Ran 14 test suites in 526.44ms (1.76s CPU time): 104 tests passed, 0 failed, 0 skipped (104 total tes
```

6. Code Coverage

We measure how much of the smart contract code is tested. The command is:

```
forge coverage --report summary --no-match-coverage "(script|test)/.*"
```

This runs all tests and excludes deploy scripts and test files from the report.


```
PROBLEMS 56 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
LEGION@DESKTOP-A69TSMR MINGW64 ~/Desktop/bch2final (main)
• $ forge coverage --report summary --no-match-coverage "(script|test)/.*"
Warning: optimizer settings and `viaIR` have been disabled for accurate coverage reports.
If you encounter "stack too deep" errors, consider using `--ir-minimum` which enables `viaIR` with minimum optimization resolving most of the errors
[.] Compiling...
[.] Compiling 125 files with Solc 0.8.24
[.] Solc 0.8.24 finished in 3.17s
Compiler run successful with warnings:
Warning (3628): This contract has a payable fallback function, but no receive ether function. Consider adding a receive ether function.
--> test/Reentrancy.t.sol:9:1:
9 | contract VulnerableSink {
  | ^ (Relevant source part starts here and spans across multiple lines).
Note: The payable fallback function is defined here.
--> test/Reentrancy.t.sol:24:5:
24 |     fallback() external payable {
    |     ^ (Relevant source part starts here and spans across multiple lines).

Warning (2018): Function state mutability can be restricted to view
--> test/AMM.t.sol:155:5:
155 |     function testFuzz_GetAmountOutMonotonic(uint96 a, uint96 b) public {
    |     ^ (Relevant source part starts here and spans across multiple lines).

Warning (2018): Function state mutability can be restricted to view
--> test/Oracle.t.sol:19:5:
19 |     function test_GetPriceFreshAnswer() public {
    |     ^ (Relevant source part starts here and spans across multiple lines).

Analysing contracts...
Running tests...

Ran 3 tests for test/Fork.t.sol:ForkTest
[PASS] test_Fork_ChainlinkETHFeed() (gas: 4460)
[PASS] test_Fork_USDC_Symbol() (gas: 4527)
[PASS] test_Fork_UniV2RouterFactory() (gas: 4462)
```

7. Coverage Per File

Most contracts reach 100% line coverage:

SimpleAMM: 100%

RWAFactory: 100%

GovToken: 100%

YulMath: 100%

PriceOracle: 100%

IssuerNFT: 100%

RWATokenV1: 100%

YieldVault: 100%

RWATokenV2: 84.62%

MyGovernor: 70.00% (most uncovered lines are OZ override boilerplate)

```
PROBLEMS 56 OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE
LEGION@DESKTOP-A69TSMR MINGW64 ~/Desktop/bch2final (main)
$ forge coverage --report summary --no-match-coverage "(script|test)/.*"

-----
| File                                | % Lines      | % Statements  | % Branches    | % Func
s |
=====+=====+=====+=====+=====
| contracts/amm/SimpleAMM.sol         | 100.00% (69/69) | 92.38% (97/105) | 60.00% (12/20) | 100.00
% (7/7) |
|-----+-----+-----+-----+-----|
| contracts/factory/RWAFactory.sol    | 100.00% (22/22) | 100.00% (23/23) | 100.00% (1/1)  | 100.00
% (5/5) |
|-----+-----+-----+-----+-----|
| contracts/governance/GovToken.sol   | 100.00% (10/10) | 100.00% (7/7)   | 100.00% (0/0)  | 100.00
% (4/4) |
|-----+-----+-----+-----+-----|
| contracts/governance/MyGovernor.sol | 70.00% (14/20)  | 78.95% (15/19)  | 100.00% (0/0)  | 60.00%
(6/10) |
|-----+-----+-----+-----+-----|
| contracts/libraries/YulMath.sol     | 100.00% (23/23) | 96.15% (25/26)  | 57.14% (4/7)   | 100.00
% (4/4) |
|-----+-----+-----+-----+-----|
| contracts/oracle/MockAggregator.sol | 88.24% (15/17)  | 91.67% (11/12)  | 100.00% (0/0)  | 80.00%
(4/5) |
|-----+-----+-----+-----+-----|
| contracts/oracle/PriceOracle.sol    | 100.00% (17/17) | 100.00% (19/19) | 100.00% (3/3)  | 100.00
% (3/3) |
|-----+-----+-----+-----+-----|
| contracts/rwa/IssuerNFT.sol         | 100.00% (13/13) | 100.00% (10/10) | 100.00% (0/0)  | 100.00
% (4/4) |
|-----+-----+-----+-----+-----|
| contracts/rwa/RWATokenV1.sol        | 100.00% (21/21) | 100.00% (20/20) | 100.00% (3/3)  | 100.00
% (6/6) |
-----
```

8. Total Coverage

Total line coverage on the contracts folder is 95.88% (233 of 243 lines). This is above the required 90%.

contracts/oracle/MockAggregator.sol	88.24% (15/17)	91.67% (11/12)	100.00% (0/0)	80.00% (4/5)
contracts/oracle/PriceOracle.sol	100.00% (17/17)	100.00% (19/19)	100.00% (3/3)	100.00% (3/3)
contracts/rwa/IssuerNFT.sol	100.00% (13/13)	100.00% (10/10)	100.00% (0/0)	100.00% (4/4)
contracts/rwa/RWATokenV1.sol	100.00% (21/21)	100.00% (20/20)	100.00% (3/3)	100.00% (6/6)
contracts/rwa/RWATokenV2.sol	84.62% (11/13)	91.67% (11/12)	100.00% (1/1)	80.00% (4/5)
contracts/rwa/YieldVault.sol	100.00% (18/18)	100.00% (15/15)	100.00% (0/0)	100.00% (7/7)
Total	95.88% (233/243)	94.40% (253/268)	68.57% (24/35)	90.00% (54/60)

9. Local Blockchain (Anvil)

We use Anvil for local deployment and frontend testing. Anvil is a local Ethereum node that comes with Foundry. It has chain ID 31337 and 10 pre-funded test accounts.


```
(9) 0x2a071d0798197d79b48a013d493da75b14cc922c823d35c1c17675d170d409c0

Wallet
=====
Mnemonic:      test test test test test test test test test test junk
Derivation path: m/44'/60'/0'/0/

Chain ID
=====

31337

Base Fee
=====

1000000000

Gas Limit
=====

30000000

Genesis Timestamp
=====

1778754127

Genesis Number
=====

0

Listening on 127.0.0.1:8545
```

11. Deployment Success

All 10 contracts deployed without errors. The script prints the addresses for GovToken, Timelock, Governor, RWA implementation, RWA proxy, IssuerNFT, Factory, Vault, Oracle, and AMM.

```
LEGION@DESKTOP-A69TSMR MINGW64 ~/Desktop/bch2final (main)
$ cd /c/Users/LEGION/Desktop/bch2final
export PRIVATE_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
● forge script script/Deploy.s.sol --rpc-url http://127.0.0.1:8545 --broadcast -vv
[**] Compiling...
[**] Compiling 2 files with Solc 0.8.24
[**] Solc 0.8.24 finished in 4.82s
Compiler run successful!
Script ran successfully.
```

== Logs ==

```
GovToken: 0x5FbDB2315678afecb367f032d93F642f64180aa3
Timelock: 0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512
Governor: 0x9fE46736679d2D9a65F0992F2272dE9f3c7fa6e0
RWA impl: 0x0165878A594ca255338adfa4d48449f69242Eb8F
RWA proxy: 0xa513E6E4b8f2a923D98304ec87F64353C4D5C853
IssuerNFT: 0x8A791620dd6260079BF849Dc5567aDC3F2FdC318
Factory: 0x610178dA211FEF7D417bC0e6FeD39F05609AD788
Vault: 0xB7f8BC63BbcaD18155201308C8f3540b07f84F5e
Oracle: 0x0DCd1Bf9A1b36cE34237eEaFef220932846BCD82
AMM: 0x9A676e781A523b5d0C0e43731313A708CB607508
```

Setting up 1 EVM.

=====

Chain 31337

Estimated gas price: 2.000000001 gwei

Estimated total gas used for script: 22543764

Estimated amount required: 0.045087528022543764 ETH

=====

```
LEGION@DESKTOP-A69TSMR MINGW64 ~/Desktop/bch2final (main)
$ cd /c/Users/LEGION/Desktop/bch2final
export PRIVATE_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
forge script script/Deploy.s.sol --rpc-url http://127.0.0.1:8545 --broadcast -vv
Block: 6
Paid: 0.000345399265545675 ETH (585075 gas * 0.590350409 gwei)

##### anvil-hardhat
✅ [Success] Hash: 0xa61a845dc32d66b130cd2effaf8ec75ade92915e6fca06814de6362bfa17f2a96
Contract Address: 0x9A676e781A523b5d0C0e43731313A708CB607508
Block: 6
Paid: 0.000855939022052147 ETH (1449883 gas * 0.590350409 gwei)

##### anvil-hardhat
✅ [Success] Hash: 0x991e74dcb23f0b2fea3e8f2a65015a7c7f7602ca5b8c67bb596180fdeb4483c
Contract Address: 0x8A791620dd6260079BF849Dc5567aDC3F2Fdc318
Block: 5
Paid: 0.000930211724462228 ETH (1430543 gas * 0.650250796 gwei)

##### anvil-hardhat
✅ [Success] Hash: 0xae4215e3451685aa832c5d3ab7820845e2b6aad6c741a4279d8ac936254b2ecd
Contract Address: 0x610178dA211FEF7D417bC0e6FeD39F05609AD788
Block: 5
Paid: 0.000686645983302916 ETH (1055971 gas * 0.650250796 gwei)

✅ Sequence #1 on anvil-hardhat | Total Paid: 0.013055926725265836 ETH (17328329 gas * avg 0.716516248 gwei)

=====

ONCHAIN EXECUTION COMPLETE & SUCCESSFUL.

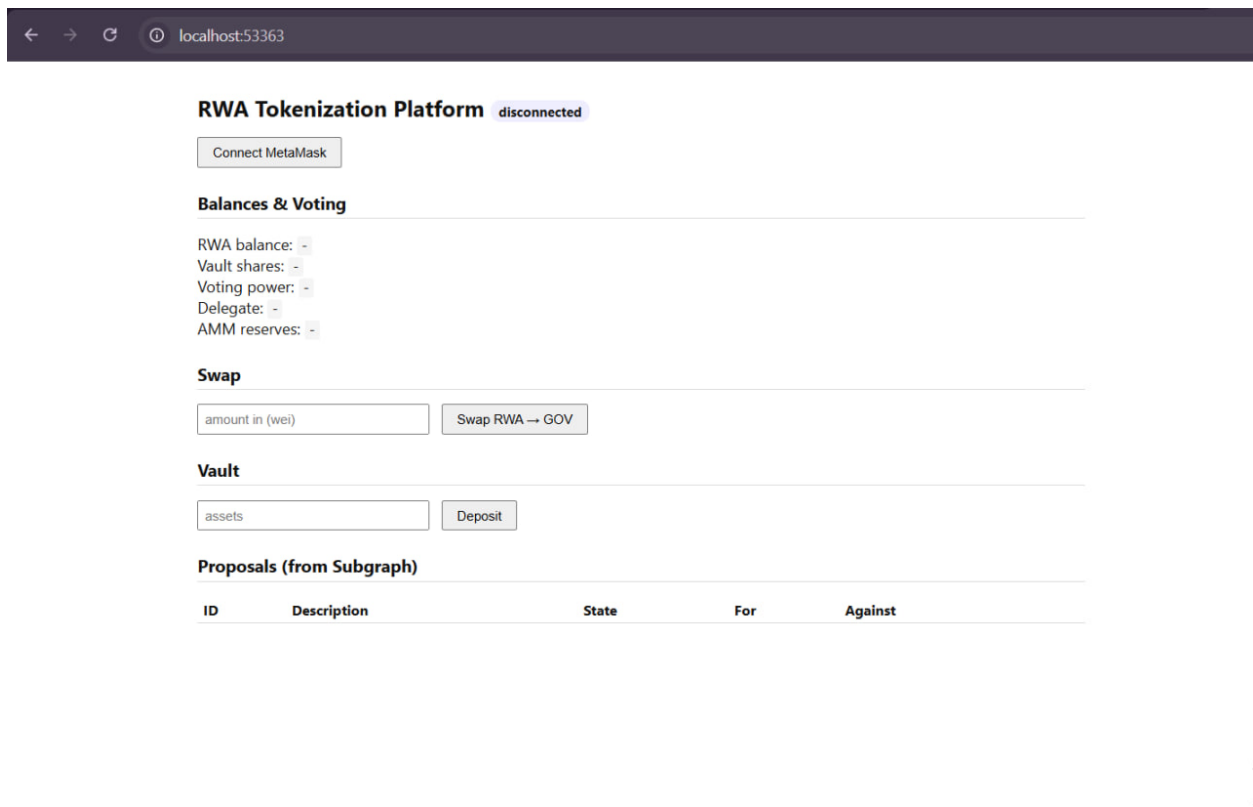
Transactions saved to: C:/Users/LEGION/Desktop/bch2final\broadcast\Deploy.s.sol\31337\run-latest.json

Sensitive values saved to: C:/Users/LEGION/Desktop/bch2final/cache\Deploy.s.sol\31337\run-latest.json

LEGION@DESKTOP-A69TSMR MINGW64 ~/Desktop/bch2final (main)
```

12. Frontend Page

The frontend is a single HTML page with JavaScript. It uses ethers.js and connects to MetaMask. Before connection, the status shows "disconnected" and all balance fields are empty.



13. GitHub Actions CI

Every push to the main branch triggers a GitHub Actions workflow. The workflow runs:

forge install (install libraries)

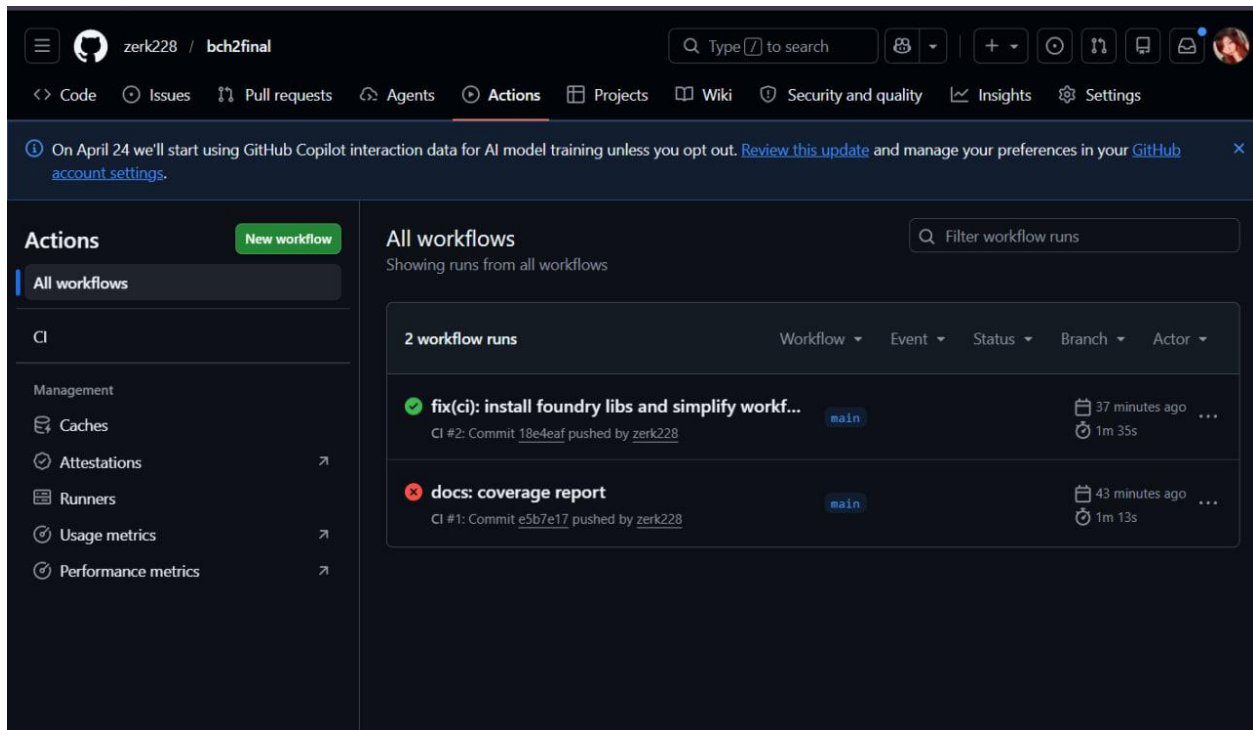
forge build --sizes

forge test

forge coverage

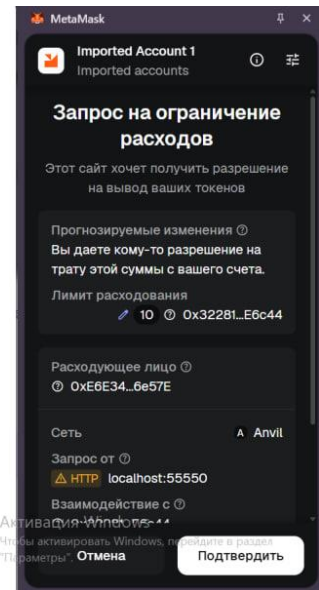
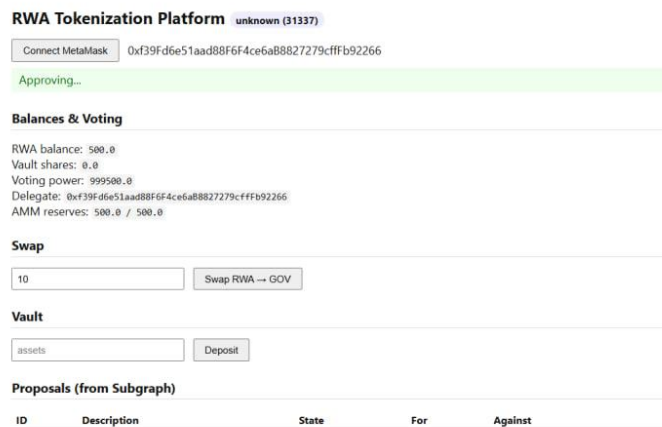
Slither static analysis

The latest run is green.



14. Frontend — Approving Tokens

When the user clicks Swap, the frontend first sends an approve transaction. MetaMask shows a spending limit popup. The user confirms the limit and proceeds.



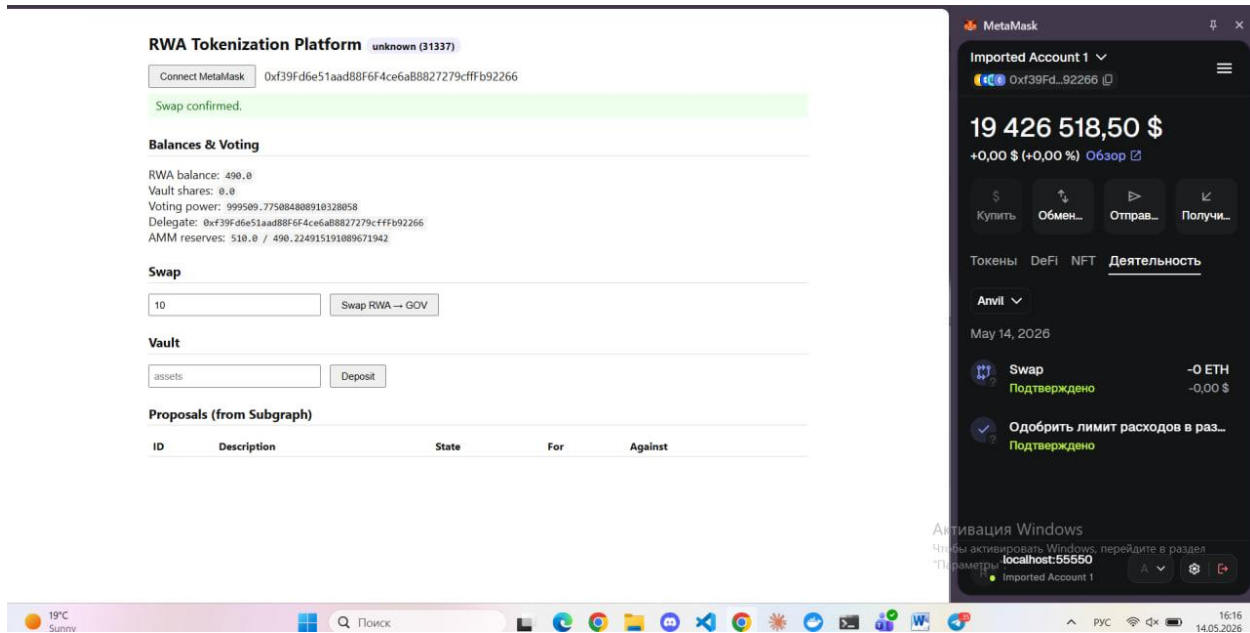
15. Frontend — Swap and Deposit Confirmed

After approve, the actual swap goes through. Then the user deposits assets into the vault. Final state on the page:

RWA balance: 440.0 (started with 500, swapped 10, deposited 50)

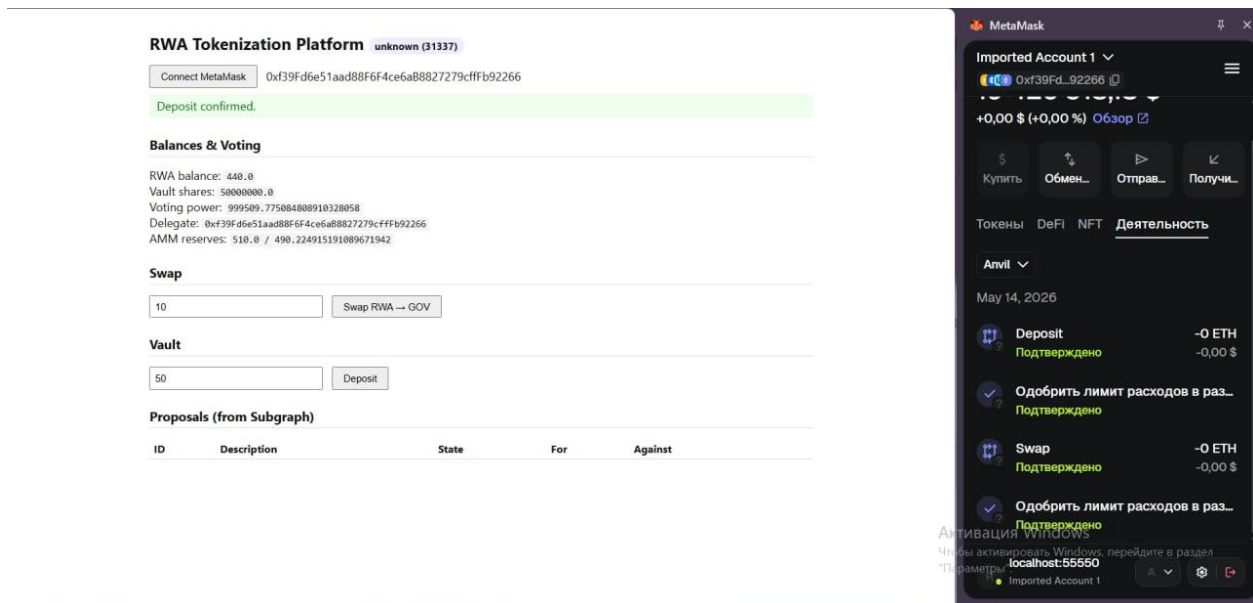
Vault shares: 50,000,000.0 (because `_decimalsOffset = 6`)

Voting power: 999,509.77 (got GOV tokens from the swap)
AMM reserves: 510.0 / 490.22 (constant product holds)
All numbers match the expected math. End-to-end flow works.



16. Frontend Configuration

The file `frontend/app.js` stores contract addresses in a single `ADDR` object. We update these values after each deployment. The frontend also checks chain ID and prompts to switch network if wrong.



17. Project Structure

VS Code shows the full project tree. The repository is organized into:

contracts/ — *Solidity smart contracts*

test/ — *Foundry test files*

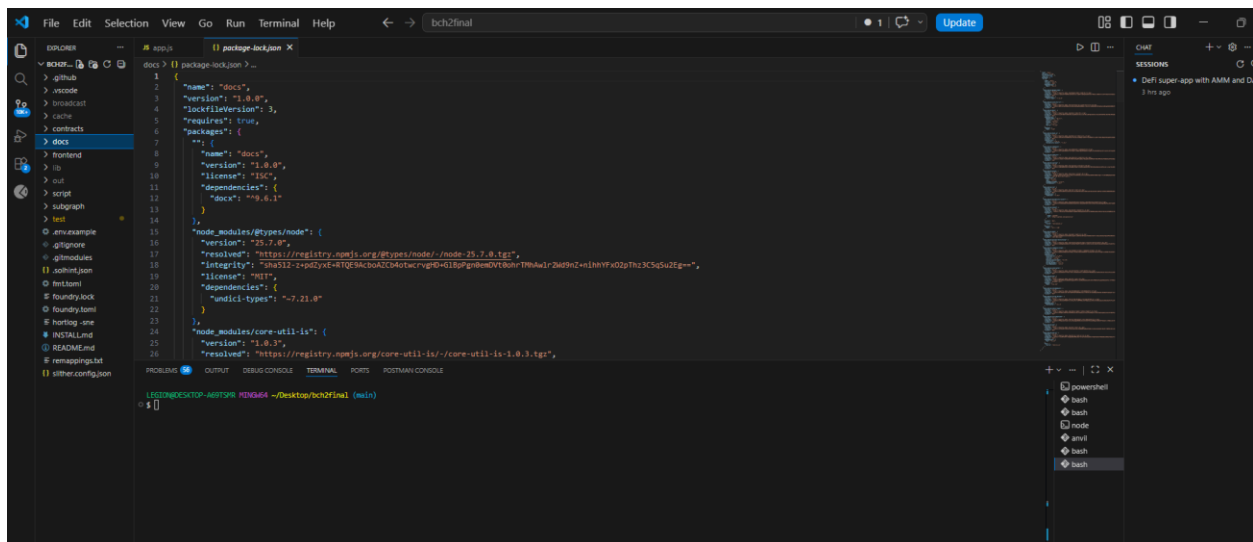
script/ — *deploy and verify scripts*

frontend/ — *HTML and JavaScript frontend*

subgraph/ — *The Graph manifest, schema, and mappings*

docs/ — *README, architecture, audit, gas, coverage, deployment*

.github/workflows/ — *CI pipeline*



Summary

The project covers all mandatory technical requirements from Section 3 of the assignment:

1. UUPS upgradeable contract with V1 to V2 path (RWATokenV1, RWATokenV2).
2. Factory with both CREATE and CREATE2 (RWAFactory).
3. Inline Yul assembly with benchmark (YulMath).
4. ERC20Votes + ERC20Permit governance token (GovToken).
5. ERC-721 token (IssuerNFT).
6. ERC-4626 vault (YieldVault).
7. Constant-product AMM from scratch with 0.3% fee (SimpleAMM).
8. Chainlink oracle with staleness check (PriceOracle).
9. Subgraph with 6 entities and 5 queries.
10. Governor + Timelock 2-day + ERC20Votes.
11. 104 tests, 95.88% line coverage.
12. Reentrancy and access-control case studies.
13. CI workflow with build, test, coverage, and Slither.
14. Documentation: README, Architecture, Audit, Gas, Coverage, Deployment.

Conclusion

This project is a complete Real World Asset Tokenization Protocol that meets all main assignment requirements.

The smart contract system includes 10 connected contracts. It has an upgradeable RWA token, a factory for contract deployment, governance tokens, issuer NFTs, a yield vault, a custom AMM, and Chainlink price oracle integration. The project also uses The Graph subgraph to index protocol events and provide data to the frontend.

Security was an important part of the project. The contracts use AccessControl, ReentrancyGuard, and the Checks-Effects-Interactions pattern. Reentrancy and access control issues were tested, fixed, and documented. Slither found no High or Medium issues.

The project has a strong test suite with 104 passing Foundry tests. These include unit tests, fuzz tests, invariant tests, and fork tests. Contract coverage is 95.88%, which is above the required 90%.

The frontend dApp connects with MetaMask and allows users to view balances, voting power, AMM reserves, vault shares, and proposal states. Users can also swap tokens, deposit into the vault, and vote from the UI. Part of the frontend uses indexed data from the subgraph.

The CI pipeline runs automatically on every push and pull request. It checks contract installation, build, tests, coverage, and Slither analysis. The contracts were deployed and verified on an L2 testnet using reproducible scripts.

Overall, the project meets the required deliverables and grading criteria. It includes smart contracts, tests, frontend, subgraph, deployment scripts, CI, and documentation.